

Part 1

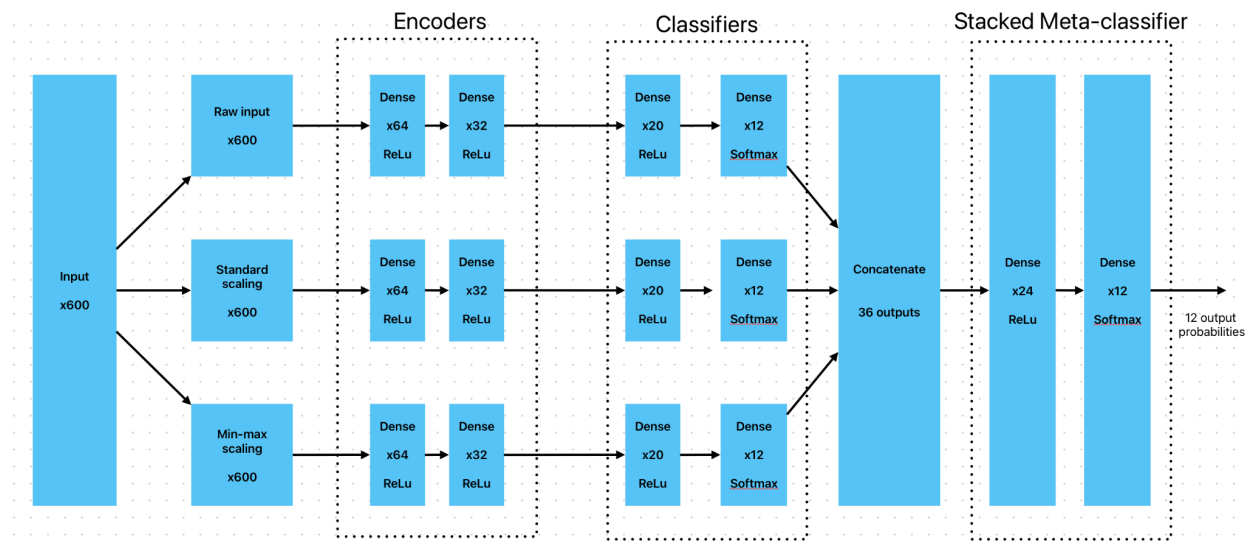
<https://studio.edgeimpulse.com/public/1008681/live>

```

Sampling...
Timing: DSP 39 ms, inference 526 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.976562
  lift: 0.023437
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 34 ms, inference 539 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
  
```

Part 2

1) Full ensemble pipeline:



2) Pruning is applied by using the Polynomial Decay schedule with initial sparsity of 0.1 and final sparsity of 0.8. The pruning wrappers are stripped before QAT because the wrappers act as weights that would be used for training, which we don't want as they are only there to show which parameters should be pruned. We want to remove them after pruning so that we go back

to a normal Keras model with sparse weights. Then, we perform QAT to this model, but use the pruning mask in `mask_callback` for the callback function when training the QAT model, to enforce the previously generated pruning decisions. The model is then compressed using int8 quantization. We need representative data for this so that we can use properly calibrated zero setpoint and scaling. This compression and pruning is vital for usage on edge devices like the Arduino Nano 33 BLE sense as they are heavily resource-constrained and we need to be able to fit the model size into the device, plus more storage for any code, and other things the device may need stored in memory.

3)

```
Class index: 7
Confidence score: 0.4375
-----
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.9219, 0.0000, 0.0000, 0.0000, 0.0664, 0.0000, 0.0000, 0.0000, 0.0078, 0.0000
Standard-scaled model scores: 0.0000, 0.0000, 0.9961, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
Min-max-scaled model scores: 0.0000, 0.0195, 0.0000, 0.0000, 0.0234, 0.0000, 0.9570, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
Meta-classifier scores: 0.0000, 0.0039, 0.3750, 0.0078, 0.0000, 0.0625, 0.0039, 0.4219, 0.0000, 0.0000, 0.0039, 0.1172
-----
Final prediction: Knees bending (crouching)
Class index: 7
Confidence score: 0.4219
-----
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.9414, 0.0000, 0.0000, 0.0000, 0.0469, 0.0000, 0.0000, 0.0000, 0.0078, 0.0000
Standard-scaled model scores: 0.0000, 0.0000, 0.9961, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
Min-max-scaled model scores: 0.0000, 0.0117, 0.0000, 0.0000, 0.0156, 0.0000, 0.9727, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
Meta-classifier scores: 0.0000, 0.0039, 0.3672, 0.0078, 0.0000, 0.0586, 0.0039, 0.4414, 0.0000, 0.0000, 0.0039, 0.1094
-----
Final prediction: Knees bending (crouching)
Class index: 7
```

The prediction of knees bending (crouching) is acceptable because from an IMU's point of view, there's no difference from a board resting in place and a human crouching in place. There is simply no movement recorded. It just depends on the board's orientation. For example, if I put the board on its side, it instead predicts "standing still". One issue I found is that when I put the board upside down and keep it still, the model predicts "jump back and forth" which doesn't make sense since the board is stationary. One change that could be made in this sense is

adding more classes in order to cover corner cases like this, so they don't get lumped into something that it obviously isn't.

4) I didn't personally notice any inconsistencies, but if there were, it could be for many reasons.

For example, difference in training data and live data from the Arduino. If these differed in magnitude, or in relative magnitudes (i.e. how much it moves in one axis relative to another during a certain motion) this could completely throw off the model because the weights wouldn't be calibrated properly.